

Aerospike Assignment

1. Install and configure 3 node Aerospike cluster community edition

Step 1.1 : ***multipass launch 20.04 --name aero-node-1 --cpus 2 --memory 4G --disk 4G***

Step 1.2 : ***multipass shell aero-node-1***

Step 1.3 : ***wget***

https://download.aerospike.com/artifacts/aerospike-server-community/7.0.0.0/aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64.tgz

What it does: This command uses *wget* to **download the compressed installation package** for the Aerospike Community Server (version 7.0.0.0, including tools) directly from the official Aerospike website.

Why: This is the first step in getting the necessary software files onto the *aero-node-1* server so you can install the database. The file specified is the correct package for the **Ubuntu 20.04** operating system running on an **aarch64** (ARM 64-bit) architecture.

Step 1.4 : ***tar -xvf***

aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64.tgz

What it does: This command uses *tar* (the standard file archive utility) to **extract (uncompress) the contents** of the downloaded *.tgz* file.

Why: The software is delivered in a compressed archive. You must extract the files to make the installation packages (*.deb* files) and other configuration files accessible on the server's file system.

Step 1.5 : ***INSTALL_DIR=\$(tar -tf aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64.tgz | head -1 | cut -d/ -f1)***

What it does: This command **extracts the name of the top-level directory** contained within the Aerospike `.tgz` file and stores it in a temporary variable named `INSTALL_DIR`.

- `tar -tf ...`: Lists the contents of the archive.
- `head -1`: Takes only the first file/directory name listed.
- `cut -d/ -f1`: Cuts the name at the first forward slash (/) and takes the first field, which is the directory name (e.g., `aerospike-server-community-7.0.0.0`).

Why: This is a programmatic way to avoid typing the long directory name manually. It guarantees you get the exact folder name created during extraction, which is needed for the next step.

Step 1.6 : ***cd \$INSTALL_DIR***

What it does: This command **changes the current working directory** to the folder name that was saved in the `INSTALL_DIR` variable (the folder created when you extracted the `.tgz` file).

Why: The installation script (`asinstall`) needed in the next step is located inside this extracted folder, so you must move into it to execute the installation.

Step 1.7 : ***sudo ./asinstall***

What it does: This command **executes the Aerospike installer script** (`asinstall`) with elevated privileges (`sudo`). This script automatically handles installing the necessary `.deb` package files for the Aerospike server and tools onto your Ubuntu system.

Why: This is the core installation step that places the Aerospike binaries, configuration files, and system service (`systemd` unit) files in the correct locations on your virtual machine.

Step 1.8 : *sudo systemctl start aerospike*

Step 1.9 : *sudo systemctl status aerospike*

```
ubuntu@aero-node-1:~/aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64$ sudo systemctl status aerospike
● aerospike.service - Aerospike Server
   Loaded: loaded (/lib/systemd/system/aerospike.service; disabled; vendor preset: enabled)
   Drop-In: /etc/systemd/system/aerospike.service.d
            ↳ aerospike.conf
   Active: active (running) since Wed 2025-12-03 17:34:19 IST; 4s ago
     Process: 22832 ExecStartPre=/usr/bin/asn-systemd-helper (code=exited, status=0/SUCCESS)
     Process: 22849 ExecStartPre=/bin/systemctl start aerospike_telemetry (code=exited, status=0/SUCCESS)
    Main PID: 22852 (asd)
       Tasks: 184 (limit: 4623)
      Memory: 253.9M
    OGroup: /system.slice/aerospike.service
           L:22852 /usr/bin/asd --config-file /etc/aerospike/aerospike.conf --fgdaemon

Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (exchange): (exchange.c:2345) data exchange started with cluster key 3e35fc14873a
Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (clustering): (clustering.c:5799) applied new succession list bb96d7bdf0065452
Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (clustering): (clustering.c:5801) applied cluster size 1
Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (exchange): (exchange.c:2728) exchange-compatibility-id: self 11 cluster-min 0 -> 11 cluster-max 0 -> 11
Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (exchange): (exchange.c:3296) received commit command from principal node bb96d7bdf0065452
Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (exchange): (exchange.c:3259) data exchange completed with cluster key 3e35fc14873a
Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (partition): (partition_balance.c:1808) (orders) replication factor is 1
Dec 03 17:34:21 aero-node-1 asd[22852]: Dec 03 2025 12:04:21 GMT: INFO (partition): (partition_balance.c:1811) (orders) rebalanced: expected-migrations (0,0,0) fresh-partitions 4896
Dec 03 17:34:22 aero-node-1 asd[22852]: Dec 03 2025 12:04:22 GMT: INFO (info): (thr_info.c:2148) Aerospike Telemetry Agent: Aerospike anonymous data collection is ACTIVE. For further information, see http://aerospike.com/aerospike-telemetry
Dec 03 17:34:22 aero-node-1 asd[22852]: Dec 03 2025 12:04:22 GMT: INFO (info): (thr_info.c:2061) latencies command: unrecognized histogram: (orders)-query
ubuntu@aero-node-1:~/aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64$ asadm
Seed: [{"127.0.0.1", 3000, None}]
Config_file: /home/ubuntu/.aerospike/astools.conf, /etc/aerospike/astools.conf
Aerospike Interactive Shell, version 2.20.0

Found 1 nodes
Online: 192.168.64.26:3000

Admin> █
```

Step 1.10 : *sudo vim /etc/aerospike/aerospike.conf*

What it does: Inside the **network** block, you configure the **heartbeat** sub-section to use specific interfaces and multicast/unicast settings.

Why: This is where you define the operational settings for the database, including networking and clustering parameters. This tells each Aerospike node how to find and confirm the presence of other nodes in the cluster, which is essential for forming a single, reliable cluster.

Step 1.11 : *sudo systemctl restart aerospike*

Step 1.12 : *Make the other two nodes similar to the first one and change the config accordingly.*

```
root@aerospike-node-2:/home/ubuntu# cat /etc/aerospike/aerospike.conf
# Aerospike database configuration file for use with systemd.

service {
    proto-fd-max 15000
    cluster-name cakery
}

logging {
    console {
        context any info
    }
}

network {
    service {
        address any
        port 3000
    }

    heartbeat {
        mode mesh
        port 3002
        address any
        mesh-seed-address-port 192.168.64.15 3002
        mesh-seed-address-port 192.168.64.16 3002
        mesh-seed-address-port 192.168.64.17 3002

        # To use unicast-mesh heartbeats, remove the 3 lines above, and see
        # aerospike_mesh.conf for alternative.

        interval 150
        timeout 10
    }

    fabric {
        port 3001
    }

    # info {
    #     port 3003
    # }
}

namespace memory {
    replication-factor 2
    default-ttl 0
    nsup-period 120
    stop-writes-sys-memory-pct 90
    storage-engine memory {
        data-size 200G
    }
}
```

Step 1.13 : *All three nodes will be able to recognize each other*

```
ubuntu@aero-node-3:~/aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64$ sudo systemctl status aerospike
● aerospike.service - Aerospike Server
   Loaded: loaded (/lib/systemd/system/aerospike.service; disabled; vendor preset: enabled)
   Drop-In: /etc/systemd/system/aerospike.service.d
           └─aerospike.conf
   Active: active (running) since Wed 2025-12-03 17:37:00 IST; 3s ago
   Process: 19988 ExecStartPre=/usr/bin/asd-systemd-helper (code=exited, status=0/SUCCESS)
   Process: 20000 ExecStartPre=/bin/systemctl start aerospike-telemetry (code=exited, status=0/SUCCESS)
   Main PID: 20000 (asd)
   Tasks: 19k (limit: 4623)
   Memory: 251.9M
   CGroup: /system.slice/aerospike.service
           └─20000 /usr/bin/asd --config-file /etc/aerospike/aerospike.conf --fgdaemon

Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (clustering): (clustering.c:5799) applied new succession list bb9c6358e005452 bb9ac867d005452 bb96d7bdf005452
Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (clustering): (clustering.c:5801) applied cluster size 3
Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (exchange): (exchange.c:2345) data exchange started with cluster key d5d3e5213491
Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (exchange): (exchange.c:2728) exchange-compatibility-id: self 11 cluster-min 0 -> 11 cluster-max 0 -> 11
Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (exchange): (exchange.c:3296) received commit command from principal node bb9c6358e005452
Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (exchange): (exchange.c:3259) data exchange completed with cluster key d5d3e5213491
Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (partition): (partition_balance.c:1808) (orders) replication factor is 2
Dec 03 17:37:02 aero-node-3 asd[20000]: Dec 03 2025 12:07:02 GMT: INFO (partition): (partition_balance.c:1811) (orders) rebalanced: expected-migrations (0,272,2838) fresh-partitions 0
Dec 03 17:37:03 aero-node-3 asd[20000]: Dec 03 2025 12:07:03 GMT: INFO (info): (thr_info.c:2148) Aerospike Telemetry Agent: Aerospike anonymous data collection is ACTIVE. For further information, see http://aerospike.com/aerospike-telemetry
Dec 03 17:37:03 aero-node-3 asd[20000]: Dec 03 2025 12:07:03 GMT: INFO (info): (thr_info.c:2081) latencies command: unrecognized histogram: {orders}-query
ubuntu@aero-node-3:~/aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64$ asadm
Seed: [{"127.0.0.1", 3000, None}]
Config file: /home/ubuntu/aerospike/astools.conf, /etc/aerospike/astools.conf
Aerospike Interactive Shell, version 2.20.0

Found 3 nodes
Online: 192.168.64.26:3000, 192.168.64.27:3000, 192.168.64.26:3000

Admin>

Admin> exit
ubuntu@aero-node-3:~/aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64$ asadm -e 'info'
Seed: [{"127.0.0.1", 3000, None}]
Config file: /home/ubuntu/aerospike/astools.conf, /etc/aerospike/astools.conf
-----Network Information (2025-12-03 12:08:39 UTC)-----
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Node | Node ID | IP | Build | Migrations | Size | Key | Integrity | Principal | Client | Uptime |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 192.168.64.26:3000 | BB96D7BDF005452 | 192.168.64.26:3000 | C-7.0.0.0 | 0.000 | 3 | D5D3E5213491 | True | BB9C6358E005452 | 13 00:04:20 |
| 192.168.64.27:3000 | *BB9C6358E005452 | 192.168.64.27:3000 | C-7.0.0.0 | 0.000 | 3 | D5D3E5213491 | True | BB9C6358E005452 | 10 00:02:47 |
| aero-node-3:3000 | BB9AC867D005452 | 192.168.64.28:3000 | C-7.0.0.0 | 0.000 | 3 | D5D3E5213491 | True | BB9C6358E005452 | 7 00:01:39 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
Number of rows: 3

-----Namespace Usage Information (2025-12-03 12:08:39 UTC)-----
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Namespace | Node | Evictions | Stop | Writes | System | Memory | Primary | Index | Secondary | Storage | Engine | Used | Avail | Avail |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| orders | 192.168.64.26:3000 | 0.000 | False | 83 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | memory | 0.000 | 0.000 | 0.000 | 0.000 |
| orders | 192.168.64.27:3000 | 0.000 | False | 83 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | memory | 0.000 | 0.000 | 0.000 | 0.000 |
| orders | aero-node-3:3000 | 0.000 | False | 84 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | memory | 0.000 | 0.000 | 0.000 | 0.000 |
| orders | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
Number of rows: 3

-----Namespace Object Information (2025-12-03 12:08:39 UTC)-----
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Namespace | Node | Rack | Repl | Expirations | Total | Records | Master | Prole | Non-Replica | Master | Prole | Non-Replica | Tombstones | Pending-Migrates |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| orders | 192.168.64.26:3000 | 0 | 2 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| orders | 192.168.64.27:3000 | 0 | 2 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| orders | aero-node-3:3000 | 0 | 2 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| orders | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
Number of rows: 3

ubuntu@aero-node-3:~/aerospike-server-community_7.0.0.0_tools-10.0.0_ubuntu20.04_aarch64$
```

We can check it through asadm command

2. The data should be persisted on disk and create a namespace orders

Step 2.1 : *sudo vim /etc/aerospike/aerospike.conf*

What it does: Opens the configuration file for editing. **Create the namespace** that will store order-related data.

Why: Namespaces are defined here to instruct the Aerospike server on how to manage data. This new namespace segregates the order data and applies the specific persistence settings required for your application.

None

Aerospike database configuration file for use with systemd.

```

service {
    proto-fd-max 15000 # This sets the maximum number of
    "File Descriptors" (open connections/files). Since every
    client connecting to the database uses a connection, 15,000
    allows for a high volume of simultaneous users.
    cluster-name cakery
}
logging {
    console {
        context any info # sends logs about every service of
        aerospike in the log files
    }
}
network {
    service {
        address any
        port 3000 // here our python/java file interact
    }
    heartbeat {
        mode mesh
        port 3002 // checking other nodes of clusters
        address any
        mesh-seed-address-port 192.168.64.26 3002
        mesh-seed-address-port 192.168.64.27 3002
        mesh-seed-address-port 192.168.64.28 3002
        # To use unicast-mesh heartbeats, remove the 3 lines
        above, and see
        # aerospike_mesh.conf for alternative.
        interval 150 #Every 150 milliseconds, this node
        sends a "ping" to the other nodes listed in your
        mesh-seed-address-port list.
        timeout 10 # the node will wait 10 sec if it did not
        recieve the ping it will consider the node dead
    }
}

```

```

    fabric {
        port 3001 #internal highway to move data between
nodes
    }
#    info {
#        port 3003
#    }
#
namespace Orders {
    replication-factor 2
    default-ttl 0
    nsup-period 120
    stop-writes-sys-memory-pct 90
    storage-engine device {
        file /var/lib/aerospike/Orders.dat
        filesize 4G
        write-block-size 128K
        defrag-lwm-pct 50
    }
}
}

```

Here is the breakdown of the Aerospike configuration parameters for the **Orders** namespace

- **replication-factor: 2**
Every piece of data in one node is stored in two other nodes on the same cluster
- **default-ttl: 0**
 - **What it does:** Sets the default **Time-To-Live (TTL)** for records to **zero (0)**.
 - **Why it's set this way:** This means records will **never expire** (i.e., they are permanent), which guarantees **data persistence**. You want the order data to remain on disk indefinitely unless explicitly deleted.
- **storage-engine device { ... }**
 - **What it does:** This is the **Crucial** command that defines the storage medium for the namespace as a block device or a file (disk).

- **Why it's set this way:** This command **enables persistence**. Without a storage-engine `device`, data would only live in memory and be lost on server restart.
- **file: /var/lib/aerospike/Orders.dat**
 - **What it does:** Specifies the **exact path and filename** on the disk where the data for the `Orders` namespace will be physically stored.
 - **Why it's set this way:** This tells Aerospike where to write the persistent data.
- **filesize: 4G**
 - **What it does:** Allocates a **maximum size of 4 Gigabytes** for the data file on disk.
 - **Why it's set this way:** This pre-allocates the storage capacity for the namespace. The index (metadata) is still kept in memory, but the bulk data is written to this file.
- **write-block-size: 128K**
 - This is the "chunk size." Aerospike writes data to the disk in 128KB blocks. This is optimized for high-speed SSD performance.

defrag-lwm-pct 50: "Defragmentation Low Water Mark." When a data block becomes 50% empty (due to deleted data), Aerospike will clean it up and consolidate it to save space.

Run `sudo systemctl restart aerospike` to Stop the current Aerospike server process and start it again. The server reads the configuration file only during startup. This action loads the new `Orders` namespace and initializes the `Orders.dat` file on disk.

Step 2.2 : `asinfo -v 'namespaces'`

What it does: Uses the `asinfo` tool to list all active namespaces.

Why: Verifies that the new `Orders` namespace has successfully been added by the server.

```
[ubuntu@aero-node-1:~$ asinfo -v 'namespaces']
Orders
ubuntu@aero-node-1:~$ █
```

Step 2.3 : `ls -l /var/lib/aerospike/`

What it does: Lists the contents of the Aerospike data directory.

Why: Confirms that the `Orders.dat` file has been created on the disk with the specified `filesize` (4G).

```
ubuntu@aero-node-1:~$ ls -l /var/lib/aerospike/  
total 8192  
-rw----- 1 aerospike aerospike 4294967296 Dec  5 12:59 Orders.dat  
ubuntu@aero-node-1:~$
```

3. Write code in python/shell which will push data into namespace orders and fetch data from namespace orders and show on webpage

Step 3.1 : ***sudo apt install python3.9 python3.9-venv***

What it does: This command uses the **apt** package manager to install important components on your Ubuntu system:

python3.9 and python3.9-venv: The Python 3.9 interpreter and the tools needed to create isolated virtual environments.

Why: You need **Python 3.9** because your previous Python 3.8 installation was too old and caused build errors when trying to install the **aerospike** client. You install **Nginx** to serve the final data output to the web browser.

Step 3.2 : ***/usr/bin/python3.9 -m venv aero-env***

What it does: This command explicitly uses the newly installed **Python 3.9** interpreter to **create a new virtual environment** named **aero-env** in your current directory.

Why: A virtual environment creates an isolated space for your project. This prevents conflicts between the **aerospike** client's dependencies and other programs installed on your main system.

Step 3.3 : *source aero-env/bin/activate*

What it does: This command **activates** the isolated environment you just created. Your command prompt will change (it will show `(aero-env)`).

Step 3.4 : *pip install --upgrade pip setuptools wheel*

What it does: This command uses the package installer (`pip`) inside your environment to update the core Python packaging tools: `pip`, `setuptools`, and `wheel`.

Why: This is a **critical fix** to avoid future installation failures (like the "invalid command 'bdist_wheel'" error). Updating these tools ensures the build process is stable and compatible with the modern standards used by the `aerospike` package.

Step 3.5 : *export*

AEROSPIKE_CLIENT_C_HOME="/home/ubuntu/aerospike-client-c_7.0.0_ubuntu20.04_x86_64"

What it does: This command **creates an environment variable** named `AEROSPIKE_CLIENT_C_HOME` and sets its value to the exact folder location where you extracted the **Aerospike C Client** files.

Why: The Python client installer needs to compile against the low-level C Client. This variable acts as a **map**, telling the C compiler exactly where to find the necessary header files and libraries, which is essential to prevent the compiler from failing.

Step 3.6 : *pip install aerospike --no-build-isolation*

Step 3.7 : *sudo vim [write.py](#)*

```
None
import aerospike
import sys
```

```

config = {'hosts': [('192.168.64.26', 3000)]}
try:
    client = aerospike.client(config).connect()
    print("--- Aerospike Interactive Data Pusher ---")
    while True:
        try:
            order_id = int(input("\nEnter Order ID (or 0 to exit): "))
            if order_id == 0: break
            cust_name = input("Enter Customer Name: ")
            item = input("Enter Item Name: ")
            price = float(input("Enter Price: "))
            # Define Key: (namespace, set, user_key)
            key = ('Orders', 'customer_orders', order_id)
            # Bins (Columns)
            bins = {
                'order_id': order_id,
                'customer_name': cust_name,
                'item': item,
                'price': price
            }
            client.put(key, bins)
            print(f"Successfully pushed Order {order_id} to Aerospike!")
        except ValueError:
            print("Invalid input. Please enter numbers for ID and Price.")
except Exception as e:
    print(f"Connection Error: {e}")
finally:
    client.close()

```

Step 3.8 : *sudo vim fetch.py*

Python

```

from flask import Flask, render_template_string
import aerospike
app = Flask(__name__)
config = {'hosts': [('192.168.64.26', 3000)]}
HTML_TEMPLATE = """
<html>

```

```

<head><title>Orders Webpage</title></head>
<body>
  <h2>Aerospike Orders Dashboard</h2>
  <table border="1">
    <tr>
      <th>Order ID</th>
      <th>Customer Name</th>
      <th>Item</th>
      <th>Price</th>
    </tr>
    {% for order in orders %}
    <tr>
      <td>{{ order['order_id'] }}</td>
      <td>{{ order['customer_name'] }}</td>
      <td>{{ order['item'] }}</td>
      <td>${{ order['price'] }}</td>
    </tr>
    {% endfor %}
  </table>
</body>
</html>
"""
@app.route('/')
def index():
    orders = []
    try:
        client = aerospike.client(config).connect()
        # Scan the entire namespace/set
        scan = client.scan('Orders', 'customer_orders')
        def callback(item):
            orders.append(item[2]) # item[2] contains the bins/data
        scan.foreach(callback)
        client.close()
    except Exception as e:
        return f"Error: {str(e)}"
    return render_template_string(HTML_TEMPLATE, orders=orders)
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Step 3.9 : ***pip install flask***

Step 3.10 : ***sudo vim /etc/nginx/sites-available/default***

```
None
server {
    listen 80;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Step 3.11 : ***sudo nginx -t***

Step 3.12 : ***sudo systemctl restart nginx***

Step 3.13 : ***python3 [write.py](#)***

Step 3.14 : ***python3 fetch.py***

Step 3.15 : ***Verification***

Verify via CLI: Using the Aerospike tool **aq1** (Aerospike Query Language) to see it from the command line, bypassing Python entirely:

- Type: **aq1**
- Run: **SELECT * FROM Orders.customer_orders**
- *This proves the data exists in the database engine independently.*

```

mysql> SELECT * FROM Orders.customer_orders
+-----+-----+-----+-----+
| order_id | customer_name | item      | price |
+-----+-----+-----+-----+
| 11       | "Shambhavi"   | "Book1"   | 250    |
| 12       | "Alice"       | "Book2"   | 260    |
+-----+-----+-----+-----+
2 rows in set (0.117 secs)

OK

mysql> █

```

Step 3.16 : **Open your browser** to your server's IP address.

Aerospike Orders Dashboard

Order ID	Customer Name	Item	Price
11	Shambhavi	Book1	\$250.0
12	Alice	Book2	\$260.0

Phase 1: The "Write" Workflow (**write.py**)

1. **Handshake:** our script initiates a connection to `192.168.64.26:3000`. The Aerospike node checks its "Bank Teller" windows (`proto-fd-max`). Since we have 15,000 available, it lets you in.
2. **The Map Check:** The script says, "I have data for `web_test_user_42`." Aerospike uses a hashing algorithm (RIPEMD-160) on that ID to decide which of your 3 nodes will be the **Master** for that record.
3. **The Safety Check:** Before writing, the node checks its health. It looks at the disk usage of the root partition.
4. **The Commit:** * The data is written to the **RAM index** (for fast searching).
 - Simultaneously, it is written to the physical file on your disk:
`/var/lib/aerospike/Orders.dat`.
5. **The Twin (Replication):** Because your config says `replication-factor 2`, the Master node sends a copy over the **Fabric Port (3001)** to a second node. Only when both nodes say "I have it!" does your script print `SUCCESS`.

Phase 2: The "Fetch" Workflow (`fetch.py`)

1. **Request:** You run `fetch.py`. It sends the "Key" tuple: (`'Orders'`, `'WebData'`, `'web_test_user_42'`).
2. **Lookup:** Aerospike doesn't search the whole disk (that would be slow). It looks at its **RAM Index** to find exactly where on the disk that record starts.
3. **Disk Read:** The "Disk Head" reads the 128K block from `Orders.dat`.
4. **Metadata:** It checks the **Generation** (how many times the record was changed) and the **TTL** (is it expired?).
5. **Delivery:** The data travels back through the "Service Port" (3000) to our Python script, which prints it to your terminal.

Simple Aerospike Architecture for Revision

Aerospike works like a highly efficient, self-managing system where all servers (nodes) are equal and work together (**shared-nothing architecture**). It breaks its job down into three specialized parts:

1. The Client Layer (The Smart App Interface)

This is the software inside your application (like a website or app) that is asking for data.

- **Job:** To find the data **instantly** and communicate the request.
- **Key Action:** When the app asks for something (e.g., "get user 50's profile"), the Client knows the **exact server** (node) where that data lives.
- **The Benefit:** It talks **directly** to the right server. This is called **zero-hop routing**, and it's the main reason Aerospike is so incredibly **fast** (low latency). It doesn't waste time talking to a central manager first.

2. The Clustering and Data Distribution Layer (The Self-Managing Robots)

This is the central intelligence that manages all the servers and keeps everything running smoothly and safely.

- **Job:** To ensure **availability**, **even workload**, and **easy scaling**.
- **Key Actions:**
 - **Data Spreading:** It uses a formula (**HPI**) to perfectly spread all the data (in fixed chunks called **partitions**) evenly across every server. Every server has the same workload (**fair load balancing**).
 - **Backup and Recovery:** It automatically keeps **copies (replicas)** of data on different servers. If one server fails, the system instantly activates the backup copy (**fail-over**).
 - **Easy Growth:** If you add a new server, the system automatically and gently **moves a small, equal portion of data** to the new server without slowing down operations (**online horizontal scaling**).
 - **Disaster Backup (XDR):** It keeps a live copy of the entire system running in a far-off location for true **Disaster Recovery**.

3. The Data Storage Layer (The Super-Fast Shelves) ⚡

This is the physical part where the information is actually kept.

- **Job:** To store data reliably and allow for the **fastest possible access**.
- **Key Actions:**

- **Uses the Best Media:** It uses **Flash SSDs** (Solid State Drives) because they are fast and reliable. It may also use regular memory (**DRAM**) for the absolute hottest, most important data (like the data map).
- **Bypassing Traffic:** It is programmed to talk **directly** to the SSD hardware, bypassing the slow traffic rules of the regular operating system. This is called **Direct I/O** or **AIO** (Asynchronous I/O).
- **SSD Friendly:** It writes data in a way that is good for the SSD's lifespan (using **append-only** writes), ensuring long-term speed and endurance.

In summary: The **App** knows where to go, the **Clustering Layer** manages availability and scaling, and the **Storage Layer** holds the data on optimized, super-fast drives.

None

```
namespace pyxis {
  enable-xdr true
  xdr-remote-datacenter DC1
  memory-size 25G
  replication-factor 2
  single-bin false
  default-ttl 0
  nsup-period 120
  high-water-disk-pct 60
  high-water-memory-pct 70
  stop-writes-pct 90
  storage-engine device {
    file /var/lib/aerospike/pyxis.dat
    filesize 100G
    compression none
    data-in-memory true
    write-block-size 128K
    defrag-lwm-pct 50
    defrag-startup-minimum 10
  }
}
```

Namespace Definition and Core Rules

- **namespace pyxis:** This is the **name** of your database container. All data and rules defined below belong to this container named **pyxis**.

- **replication-factor: 2**: Every record in this container will have **two identical copies** stored across the cluster nodes. This ensures **high availability** if one server fails.
- **default-ttl: 0**: **TTL means Time-To-Live** (how long a record lasts). Setting it to **0** means records will **never expire** (i.e., they are permanent) unless you delete them manually.
- **single-bin false**: This allows you to store records with **multiple "bins"** (which are like columns or fields), which is the standard, flexible way to use Aerospike.
- **nsup-period: 120**: Sets how often (every **120 seconds**) the background cleanup process runs to look for and remove any expired records (though none will expire due to **default-ttl 0**).

Performance and Protection Limits

These settings help Aerospike manage its resources and prevent the cluster from crashing.

- **memory-size: 25G**: Sets aside **25 Gigabytes of RAM** (memory) specifically for storing the **index and metadata** (the pointers that track where your data is on the disk).
- **data-in-memory: true**: **IMPORTANT**: Even though the main data is stored on disk (see below), this setting tells Aerospike to **keep a copy of the data in RAM** as well. This makes data retrieval extremely fast.
- **high-water-memory-pct: 70**: If memory usage reaches **70%** of the allocated 25GB, Aerospike will take corrective action, like forcing cleanup, to manage the load.
- **high-water-disk-pct: 60**: Sets a **"warning" threshold** for disk usage at **60%** of the total allocated disk space.
- **stop-writes-pct: 90**: Sets a **critical limit** at **90%** of the memory/disk usage. If usage hits this level, Aerospike will **stop accepting new write requests** to prevent the node from running out of resources and failing.

Disk Persistence and Storage (**storage-engine device**)

This section is where you tell Aerospike to use the hard drive for permanent storage.

- **storage-engine device { ... }**: This command **enables Disk Persistence**. It tells Aerospike to use your device (hard drive) to store the main record data, making it permanent.
- **file: /var/lib/aerospike/pyxis.dat**: Specifies the **exact file path** on the disk where the data for **pyxis** will be physically saved.
- **filesize: 100G**: **Pre-allocates 100 Gigabytes** of space for this data file on the disk.

- **write-block-size: 128K**: Defines the size of the blocks (chunks) Aerospike uses when writing data to the disk file.

Cross-Datacenter Replication (XDR)

- **enable-xdr: true**: Turns on Cross-Datacenter Replication (XDR). This tells Aerospike that data in this container should be copied to a remote location automatically.
- **xdr-remote-datacenter: DC1**: Specifies the **name of the other datacenter** (DC1) where the data will be copied for backup and disaster recovery.